JavaScript: array-ak eta metodo nagusiak

Zer da array bat?

Array bat hainbat balio ordenatuta gordetzeko erabiltzen den datu-egitura da.

```
const frutas = ["sagarra", "madaria", "platanoa"];
```

Elementu bakoitzak posizio bat edo **indize** bat dauka. Indizea 0tik hasten da:

```
frutas[0]; // "sagarra"
  frutas[1]; // "madaria"
  frutas[2]; // "platanoa"
```

Metodoen sailkapena

Array-en metodoak ulertzeko oso garrantzitsua da jakitea ea **jatorrizko array-a aldatzen duten** ala **array berri bat sortzen duten**.

Taldea	Metodoak
Jatorrizko array-a ALDATZEN dutenak	`push()`, `pop()`, `shift()`, `unshift()`, `splice()`, `sort()`, `reverse()`
Array BERRIA itzultzen dutenak	`map()`, `filter()`, `slice()`
Elementuak bilatu edo baldintzak egiaztatzen dituztenak	`find()`, `findIndex()`, `includes()`, `some()`, `every()`
Array-a zeharkatu edo beste formatu batera bihurtzen dutenak	`forEach()`, `join()`

Array-aren amaieran elementuak gehitzea: push()

`push()`-ek elementu bat edo gehiago gehitzen ditu array-aren amaieran.

```
const frutas = ["sagarra", "madaria"];

  frutas.push("platanoa");

  console.log(frutas);
  // ["sagarra", "madaria", "platanoa"]
```

Elementu bat baino gehiago ere gehitu daitezke:

```
frutas.push("kiwia", "laranja");
```

Garrantzitsua

`push()`-ek **jatorrizko array-a aldatzen du**.

ALDATZEN DU: bai
ITZULTZEN DU: array-aren luzera berria

Azken elementua kentzea: pop()

`pop()`-ek array-aren azken elementua kentzen du.

```
const frutas = ["sagarra", "madaria", "platanoa"];

  const fruta = frutas.pop();

  console.log(fruta);
  // "platanoa"

  console.log(frutas);
  // ["sagarra", "madaria"]
```

ALDATZEN DU: bai
ITZULTZEN DU: kendutako elementua

Hasieran elementuak gehitzea: unshift()

`unshift()`-ek elementuak array-aren hasieran gehitzen ditu.

```
const frutas = ["madaria", "platanoa"];

  frutas.unshift("sagarra");

  console.log(frutas);
  // ["sagarra", "madaria", "platanoa"]
```

ALDATZEN DU: bai
ITZULTZEN DU: array-aren luzera berria

Lehen elementua kentzea: shift()

`shift()`-ek lehen elementua kentzen du.

```
const frutas = ["sagarra", "madaria", "platanoa"];

  const fruta = frutas.shift();

  console.log(fruta);
  // "sagarra"

  console.log(frutas);
  // ["madaria", "platanoa"]
```

ALDATZEN DU: bai
ITZULTZEN DU: kendutako elementua

splice(): elementuak ezabatu, gehitu edo ordezkatu

`splice()` oso metodo erabilgarria da. Array baten erdian elementuak **ezabatu, gehitu edo ordezkatu** ditzake.

Sintaxia:

```
array.splice(hasiera, ezabatuKopurua, elementua1, elementua2, ...);
```

Elementuak ezabatu

```
const frutas = ["sagarra", "madaria", "platanoa", "kiwia"];

  frutas.splice(1, 2);

  console.log(frutas);
  // ["sagarra", "kiwia"]
```

- 1: bigarren elementutik hasten da.
- 2: bi elementu ezabatzen ditu.

Elementuak gehitu

0 jartzen badugu ezabatu beharreko elementu kopuru gisa:

```
const frutas = ["sagarra", "platanoa"];

  frutas.splice(1, 0, "madaria");

  console.log(frutas);
  // ["sagarra", "madaria", "platanoa"]
```

Elementuak ordezkatu

```
const frutas = ["sagarra", "madaria", "platanoa"];

  frutas.splice(1, 1, "laranja");

  console.log(frutas);
  // ["sagarra", "laranja", "platanoa"]
```

Garrantzitsua

`splice()`-ek **jatorrizko array-a aldatzen du**.

Gainera, ezabatutako elementuak array batean itzultzen ditu:

```
const frutas = ["sagarra", "madaria", "platanoa"];

  const eliminadas = frutas.splice(1, 1);
```

```
  console.log(eliminadas);
  // ["madaria"]
```

ALDATZEN DU: bai
ITZULTZEN DU: ezabatutako elementuen array-a

slice(): array baten zati bat kopiatu

`slice()`-ek array baten zati bat hartzen du eta **array berri bat** sortzen du.

```
const frutas = ["sagarra", "madaria", "platanoa", "kiwia"];

  const algunas = frutas.slice(1, 3);

  console.log(algunas);
  // ["madaria", "platanoa"]

  console.log(frutas);
  // ["sagarra", "madaria", "platanoa", "kiwia"]
```

`slice(1, 3)`: 1. posiziotik 3. posiziora arte, baina 3.a sartu gabe.

Garrantzitsua

ALDATZEN DU: ez
ITZULTZEN DU: array berria

map(): elementu guztiak eraldatu

`map()`-ek array bateko elementu bakoitza eraldatu eta **array berri bat** sortzen du.

Adibidez, prezio guztiak bikoiztu:

```
const precios = [10, 20, 30];

  const resultado = precios.map(precio => precio * 2);

  console.log(resultado);
  // [20, 40, 60]
```

```
  console.log(precios);
  // [10, 20, 30]
```

Kontzeptua:

```
10 → 20
  20 → 40
  30 → 60
```

`map()` erabiltzen dugu:

> *Elementu bakoitza beste zerbait bihurtu nahi dugunean.*

Beste adibide bat:

```
const nombres = ["Ane", "Jon", "Mikel"];

  const saludos = nombres.map(nombre => "Kaixo " + nombre);

  console.log(saludos);
  // ["Kaixo Ane", "Kaixo Jon", "Kaixo Mikel"]
```

ALDATZEN DU: ez
ITZULTZEN DU: array berria

filter(): baldintza betetzen duten elementuak aukeratu

`filter()`-ek baldintza bat betetzen duten elementuak aukeratzen ditu eta **array berri bat** sortzen du.

```
const edades = [12, 18, 25, 15, 30];

  const mayores = edades.filter(edad => edad >= 18);

  console.log(mayores);
  // [18, 25, 30]
```

Kontzeptua:

> *Mantendu baldintza betetzen duten elementuak.*

Beste adibide bat:

```
const precios = [20, 50, 120, 80, 200];

  const baratos = precios.filter(precio => precio < 100);

  console.log(baratos);
  // [20, 50, 80]
```

ALDATZEN DU: ez
ITZULTZEN DU: array berria

map() eta filter() arteko aldea

`map()`	elementuak ERALDATU
`filter()`	elementuak AUKERATU

find(): elementu bat bilatu

`find()`-ek baldintza bat betetzen duen **lehen elementua** itzultzen du.

```
const usuarios = [
      { id: 1, nombre: "Ane" },
      { id: 2, nombre: "Jon" },
      { id: 3, nombre: "Mikel" }
  ];

  const usuario = usuarios.find(usuario => usuario.id === 2);

  console.log(usuario);
  // { id: 2, nombre: "Jon" }
```

Ez badu aurkitzen:

```
undefined
```

ALDATZEN DU: ez
ITZULTZEN DU: elementu bat

findIndex(): elementu baten posizioa bilatu

`findIndex()`-ek baldintza betetzen duen lehen elementuaren indizea itzultzen du.

```
const usuarios = [
      { id: 1, nombre: "Ane" },
      { id: 2, nombre: "Jon" },
      { id: 3, nombre: "Mikel" }
  ];

  const posicion = usuarios.findIndex(usuario => usuario.id === 2);

  console.log(posicion);
  // 1
```

Ez badu aurkitzen:

```
-1
```

ALDATZEN DU: ez
ITZULTZEN DU: indize bat

includes(): elementu bat dagoen egiaztatu

`includes()`-ek elementu bat array-an dagoen egiaztatzen du.

```
const frutas = ["sagarra", "madaria", "platanoa"];

  console.log(frutas.includes("madaria"));
  // true

  console.log(frutas.includes("kiwia"));
  // false
```

ALDATZEN DU: ez
ITZULTZEN DU: `true` edo `false`

indexOf(): elementu baten indizea bilatu

```
const frutas = ["sagarra", "madaria", "platanoa"];

  console.log(frutas.indexOf("madaria"));
  // 1

  console.log(frutas.indexOf("kiwia"));
  // -1
```

ALDATZEN DU: ez
ITZULTZEN DU: indizea edo `-1`

some(): elementuren batek baldintza betetzen du?

`some()`-ek egiaztatzen du gutxienez **elementu batek** baldintza betetzen duen.

```
const edades = [12, 15, 17, 20];

  const hayAdultos = edades.some(edad => edad >= 18);

  console.log(hayAdultos);
  // true
```

Galdera hau egiten du:

> *Gutxienez batek betetzen al du baldintza?*

ALDATZEN DU: ez
ITZULTZEN DU: `true` edo `false`

every(): elementu guztiek baldintza betetzen dute?

`every()`-ek egiaztatzen du **elementu guztiek** baldintza betetzen duten.

```
const edades = [20, 25, 30];

  const todosAdultos = edades.every(edad => edad >= 18);
```

```
  console.log(todosAdultos);
  // true
```

Baina:

```
const edades = [20, 15, 30];

  const todosAdultos = edades.every(edad => edad >= 18);

  console.log(todosAdultos);
  // false
```

Galdera hau egiten du:

> *Guztiek betetzen al dute baldintza?*

forEach(): array bateko elementu guztiak zeharkatu

`forEach()`-ek elementu bakoitzeko kode bat exekutatzen du.

```
const frutas = ["sagarra", "madaria", "platanoa"];

  frutas.forEach(fruta => {
      console.log(fruta);
  });
```

Emaitza:

```
sagarra
  madaria
  platanoa
```

`forEach()` eta `map()` ez dira gauza bera.

```
const resultado = frutas.forEach(fruta => {
      console.log(fruta);
  });

  console.log(resultado);
  // undefined
```

`map()`-ek array berria sortzen du; `forEach()`-ek ez.

`forEach()`	elementu bakoitzarekin zerbait egin
`map()`	elementu bakoitza eraldatu eta array berria sortu

join(): array-a testu bihurtu

`join()`-ek array bateko elementuak testu bakarrean elkartzen ditu.

```
const frutas = ["sagarra", "madaria", "platanoa"];

  const texto = frutas.join(", ");

  console.log(texto);
  // "sagarra, madaria, platanoa"
```

Beste adibide bat:

```
const palabras = ["Kaixo", "mundua"];

  console.log(palabras.join(" "));
  // "Kaixo mundua"
```

ALDATZEN DU: ez
ITZULTZEN DU: string bat

sort(): ordenatu

`sort()`-ek array bateko elementuak ordenatzen ditu.

```
const nombres = ["Mikel", "Ane", "Jon"];

  nombres.sort();

  console.log(nombres);
  // ["Ane", "Jon", "Mikel"]
```

Zenbakiekin kontuz

```
const numeros = [10, 2, 30, 5];

  numeros.sort();

  console.log(numeros);
  // [10, 2, 30, 5]
```

`sort()`-ek lehenespenez testu bezala ordenatzen ditu.

Zenbakizko ordena lortzeko:

```
numeros.sort((a, b) => a - b);

  console.log(numeros);
  // [2, 5, 10, 30]
```

Handienetik txikienera:

```
numeros.sort((a, b) => b - a);
```

Garrantzitsua

`sort()`-ek **jatorrizko array-a aldatzen du**.

reverse(): ordena alderantzikatu

```
const numeros = [1, 2, 3, 4];

  numeros.reverse();

  console.log(numeros);
  // [4, 3, 2, 1]
```

`reverse()`-ek jatorrizko array-a aldatzen du.

Metodoen laburpena

Jatorrizko array-a ALDATZEN dute

Metodoa	Zer egiten du
`push()`	amaieran gehitu
`pop()`	amaieratik kendu
`unshift()`	hasieran gehitu
`shift()`	hasieratik kendu
`splice()`	ezabatu / gehitu / ordezkatu
`sort()`	ordenatu
`reverse()`	alderantzikatu

ARRAY BERRIA sortzen dute

Metodoa	Zer egiten du
`map()`	elementuak eraldatu
`filter()`	elementuak iragazi
`slice()`	zati bat kopiatu

Adibidez:

```
const precios = [10, 20, 30, 40];

  const resultado = precios
      .filter(precio => precio >= 20)
      .map(precio => precio * 1.21);

  console.log(resultado);
  // [24.2, 36.3, 48.4]
```

Prozesua:

Pausoa	Balioa
Hasierako array-a	`[10, 20, 30, 40]`
`filter()`	`[20, 30, 40]`

Pausoa	Balioa
`map()`	`[24.2, 36.3, 48.4]`

Array originala:

```
console.log(precios);
  // [10, 20, 30, 40]
```

ez da aldatu.

Bilatu edo egiaztatu

Metodoa	Zer egiten du
`find()`	lehen elementua
`findIndex()`	lehen elementuaren indizea
`includes()`	elementua dago? true/false
`indexOf()`	elementuaren indizea
`some()`	baten batek betetzen du? true/false
`every()`	guztiek betetzen dute? true/false

Zeharkatu edo bihurtu

Metodoa	Zer egiten du
`forEach()`	elementu bakoitzarekin ekintza bat
`join()`	array-a string bihurtu

Ideia garrantzitsuena

Array-en metodoak ikasteko ez da nahikoa sintaxia buruz ikastea.

Lehenengo galdetu:

Zer egin nahi dut?

Zer egin nahi dut?	Metodoa
Elementu bat gehitu?	`push()`
Elementu bat kendu?	`pop() / shift()`
Elementuak ordezkatu?	`splice()`
Elementu guztiak eraldatu?	`map()`
Batzuk aukeratu?	`filter()`
Bat bilatu?	`find()`
Baten bat dagoen egiaztatu?	`some()`
Guztiek baldintza betetzen dute?	`every()`

Eta bigarren galdera:

Jatorrizko array-a aldatu nahi dut?

Erantzuna	Metodoak
BAI	`push, pop, shift, unshift, splice, sort, reverse`
EZ	`map, filter, slice`

Horrek JavaScript-en array-ak ulertzeko oinarri oso ona ematen du.